

Day3 · 17차시

# LLMOps 개요 & 라이프사이클

만드는 것에서 운영하는 것으로 — 배포 버튼을 누른 다음 날부터가 진짜 시작.  
day2에서 키운 에이전트를, 이제 계속 살아 있게 만든다. (+ Langfuse 관찰성)

SECTION 00

# 오프닝 — 데모는 쉽고, 유지가 어렵다

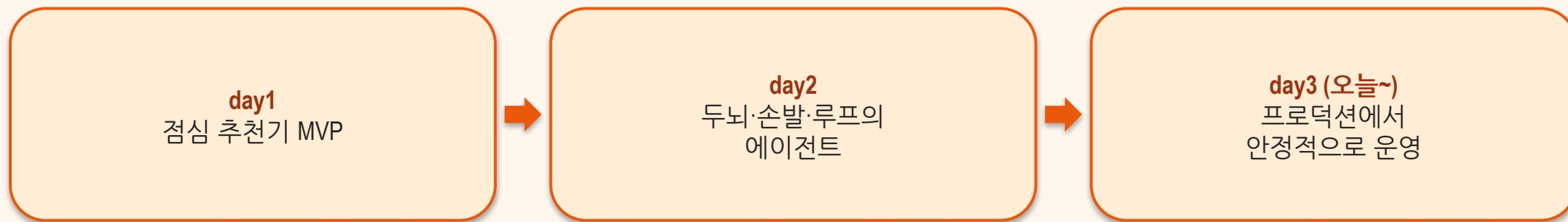
프로덕션에 올린 다음에 일어나는 일들

## “LLM 앱을 프로덕션에 올린 다음, 무슨 일이 일어나는가?”

— 책 LLMOps의 핵심 질문 — 데모는 하루, 운영은 매일

## 우리가 만든 에이전트를 다시 본다

▶ 같은 MVP, 이번엔 무너지지 않게 — 저녁 팀 프로젝트에 바로 이어진다



## LLM 프로젝트가 무너지는 3가지 이유 (Gartner 2024)

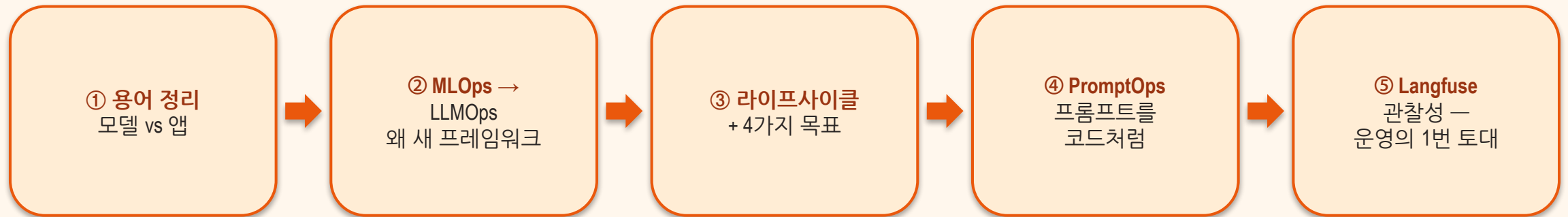
▶ 모델이 나빠서가 아니다 — 운영이 없어서다

**데이터 품질**  
들어가는 데이터가  
오염되면 답도 오염

**평가 프레임워크 부재**  
좋아졌는지 나빠졌는지  
아무도 증명 못 함

**스케일 비용**  
사용자가 늘수록  
추론 비용이 폭발

## 이 차시 동안 얻는 것



SECTION 01

# 먼저 용어부터 — 무엇을 운영하나

## 파운데이션 모델 ⊃ 생성형 AI ⊃ LLM

▶ 포함 관계로 외우자 — DALL·E는 GenAI지만 LLM은 아니다

파운데이션 모델 (토대)

대규모 데이터로 사전학습된 범용 모델 — 이미지·음성·코드 등

생성형 AI / GenAI (콘텐츠 생성)

새 콘텐츠를 만든다 — DALL·E(이미지)도 여기

LLM (언어 특화)

텍스트를 이해·생성 — GPT·Claude·Gemini·Llama



## 모델 ≠ 앱

▶ 우리가 운영하는 것은 '모델'이 아니라 '앱' — 좋은 모델을 골랐다고 끝이 아니다

### 모델 (가중치/API)

- GPT-4o · Claude · Gemini · Llama
- 가중치 또는 API 엔드포인트
- 입력→출력, 그 자체로는 '부품'

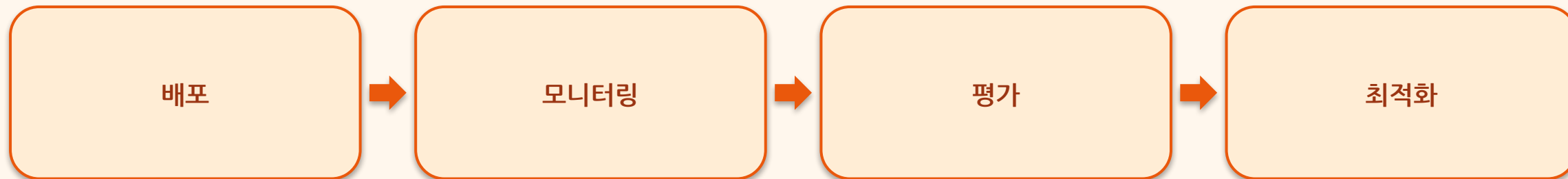
VS

### 앱 (사용자가 쓰는 것)

- ChatGPT · 우리 MVP
- 지시문·맥락·도구·안전장치를 덧댐
- RAG·메모리·가드레일 포함
- API 직접 호출과 응답이 다르다

## LLMOps란 무엇인가

▶ 사람·프로세스·기술을 정렬해 LLM 앱을 안전·견고·신뢰 있게 장기 운영하는 일



🔥 불을 계속 켜 두기 (keep the lights on) — 배포는 끝이 아니라 시작

SECTION 02

# MLOps → LLMOps — 왜 새 프레임워크인가

## 운영 프레임워크의 계보

▶ 새로 발명이 아니라 계승 + 확장

1

**Lean / Six Sigma (1986)**

제조 품질·낭비 제거의 원칙

2

**DevOps (2008)**

개발과 운영의 통합 — CI/CD·자동화

3

**MLOps (2018)**

머신러닝 모델의 학습·배포·모니터링

4

**LLMOps (2023~)**

LLM 특성에 맞춘 운영 — 우리가 지금 배우는 것

## MLOps vs LLMOps

▶ 규모 비유: MLOps=단독주택, LLMOps=부르즈 할리파 — 전체 파인튜닝은 비현실적

### MLOps

- 판별형·예측 모델
- 수동적 소비 (대시보드·추천)
- 자체 데이터로 모델 학습
- 평가가 비교적 명확 (정답 라벨)

VS

### LLMOps

- 생성형 모델 — 열린 출력
- 능동적 상호작용 (Software 3.0)
- 대부분 프롬프트·RAG로 적응
- 거대 모델을 API로 빌려 씀 — DevOps에 가깝다

## LLMOps 성숙도 — Level 0 → 2

▶ 우리 캠프의 목표는 Level 1.5 — 트레이싱·평가·프롬프트 관리가 있는 상태

### Level 0 — 없음

수동·임기응변 — 배포하면 끝, 문제는 사용자 제보로 발견

### Level 1 — MLOps 이식

문서화·모니터링은 있지만 LLM 특화(프롬프트 관리·드리프트) 누락

### Level 2 — 본격 LLMOps

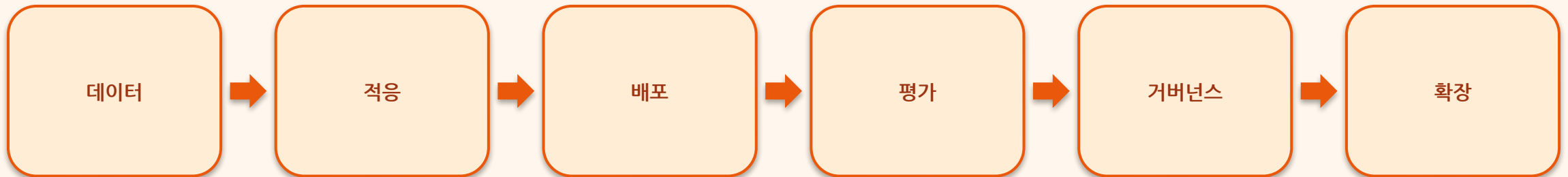
자동 모니터링 + 사람 검토 루프 + 컴플라이언스 + 레드팀

SECTION 03

## LLMOps 라이프사이클 — 운영의 수명주기

## 라이프사이클은 한 방향이 아니다

▶ 적응 = 프롬프트/RAG/PEFT · 거버넌스 = 모니터링·보안 — 각 단계는 Day3 남은 차시에서



🔄 평가 결과로 데이터·프롬프트를 고쳐 다시 — 끝없는 반복 루프



## LLMOps가 지키는 4가지

▶ 반복 작업을 자동화해 “LLM-oops!” 순간을 피하는 것이 팀의 일

### 보안 (Security)

프롬프트 인젝션·  
데이터 유출 방어

### 확장성 (Scalability)

트래픽·비용을  
감당하며 커지기

### 견고성 (Robustness)

이상 입력·장애에도  
버티기

### 신뢰성 (Reliability)

일관되고 정확한  
응답 유지

## SLO · SLA · KPI — 기술을 비즈니스 언어로

▶ 4목표를 '측정 가능한 약속'으로 바꾸는 세 층

**SLO(Service Level Objective) — 내부 목표**

우리 팀의 지향 목표 — 예: 가동률 99.9%, 지연 <200ms

**SLA(Service Level Agreement) — 외부 약속**

고객과의 계약 — 어기면 보상이 따르는 보증

**KPI(Key Performance Index) — 비즈니스 지표**

이탈률·CSAT>90% — 경영진이 읽는 숫자

## 파인튜닝이 전부가 아니다

▶ GPT-4급 하이퍼파라미터 공간  $\approx$  BERT의 1,500배 — 싼 방법 먼저

### 비용↓ 먼저 시도

- 프롬프트 엔지니어링
- RAG (검색 증강)
- 지식 그래프
- PEFT (부분 파인튜닝)

VS

### 비용↑ 마지막 수단

- 전체 파인튜닝
- 막대한 데이터·연산
- 우리 MVP는 프롬프트+RAG로 충분

## 빌드와 유지의 책임 분리 (Software 3.0)

▶ 성숙한 도구가 적어 조직마다 오픈소스를 조합 — 그래서 Langfuse 같은 오픈소스가 중요

### 만든다 (build)

- LLM / AI 엔지니어
- 데이터·리서치 사이언티스트
- 기능·프롬프트·RAG 설계

VS

### 유지한다 (maintain)

- LLMOps 엔지니어
- SW 엔지니어 · 비즈니스 운영
- 모니터링·평가·비용 관리

## LLMOps 팀의 네 역할

▶ 한 사람이 전부 못 한다 — 학제간 팀이 기본 (우리 팀 프로젝트에선 역할을 겸한다)

### 데이터 엔지니어

청킹·토크나이즈  
데이터 파이프라인

### AI 엔지니어

풀스택 개발  
프롬프트·PoC

### ML 사이언티스트

알고리즘·아키텍처  
파인튜닝

### LLMOps 엔지니어

인프라·신뢰성  
모니터링·비용

SECTION 04

# PromptOps — 프롬프트를 자산으로

# 프롬프트를 코드처럼 다루기

▶ '한 줄 바꿨더니 품질이 무너졌다'를 막는 안전장치 — 상세는 19차시에서

1

## 작성 (Author)

역할·범위·예시를 명시해 프롬프트 작성

2

## 버전 태깅 (Version)

흩어진 문자열이 아니라 1급 자산 — 버전을 붙인다

3

## 평가 → 배포 (Eval → Deploy)

바꾸기 전에 테스트, 통과하면 배포·롤백 가능하게

4

## 모니터링 → 개선

운영 데이터로 다시 개선 (루프) — 원재료는 프롬프트-응답 로깅

SECTION 05

## [관찰성] Langfuse — 왜 트레이싱이 먼저인가

4목표·평가·PromptOps 모두 '무슨 일이 있었는지 보이는 것' 위에서만 가능



## 블랙박스 vs 유리상자

▶ 비결정적·다단계 LLM 앱에서는 '재현'이 어렵다 → 기록(trace)이 유일한 진실

### 블랙박스 (로그 없음)

- 무엇이 들어갔는지 모름
- 왜 그 답이 나왔는지 모름
- 비용·지연을 추정만 함
- 문제 생기면 재현 불가

VS

### 유리상자 (트레이스)

- 입력·출력이 단계별로 보임
- 도구 호출·중간 추론이 보임
- 토큰·비용·지연이 숫자로
- 문제를 그 자리에서 추적

## Langfuse — 오픈소스 LLM 관찰성 플랫폼

▶ 계측은 @observe() 한 줄 — OpenAI 래퍼로 토큰·비용 자동

**트레이싱**  
입력·출력·지연 기록

**평가 (Score)**  
트레이스에 품질 점수

**프롬프트 관리**  
버전·배포 (19차시)

**대시보드**  
비용·지연 한눈에

오픈소스 · Self-host 가능 — '조직마다 자체 도구를 만드는' 공백을 메운 사실상의 출발점

## Session ⊃ Trace ⊃ Observation (+ Score)

▶ 이 4개념만 잡으면 대시보드가 한눈에 읽힌다 — Score는 평가(20차시)와 직접 연결

**Session** (대화 묶음)

여러 요청을 하나의 사용자 대화로 묶음

**Trace** (요청 1건)

사용자 요청 한 건의 전체 처리

**Observation** (내부 단계)

span / generation — 그 안의 각 단계

**Score** (품질 점수)

trace·observation에 붙이는 평가 점수

# 트레이스 하나에 4개념이 다 보인다

▶ 로컬 Langfuse — Trace(맞집 추천 요청) ⊃ Observation(벡터 검색·답변 생성) + Score(groundedness·relevance·👍)

The screenshot displays the Langfuse web interface for a specific trace. The left sidebar contains navigation links: Home, Dashboards, Observability, Tracing (selected), Sessions, Users, Prompt Management, Prompts, Playground, Evaluation, Scores, Evaluators, Human Annotation, and Datasets. A 'Star Langfuse' notification is also present.

The main trace view shows the following details:

- Trace ID:** 6201a9d369ac0f03dee0a843bfe98f8c
- Match Request:** 2.16s, \$0.000134
- Observation:** groundedness: 0.88, relevance: 0.95
- Input:** question: "늦게까지 하는 해장국집"
- Output:** '늦게까지 하는 해장국집' 요청에 맞춰 3곳 추천드려요. 1. 정돈 — 분위기가 좋아 만족도가 높아요 [1], 2. 우래옥 — 가성비와 접근성이 좋습니다 [2], 3. 성수룩발 — 재방문 의사가 높은 곳이에요 [3]
- Metadata:** user: "user-daniel", session: "sess-late-08", round: 3, telemetry.sdk.language: "python", telemetry.sdk.name: "opentelemetry", telemetry.sdk.version: "1.43.0", service.instance.id: "bdbdf9e4-9e8c-473f-8d8d-e0d7dc366d21", service.name: "unknown\_service", scope: "langfuse-sdk", version: "4.13.2"

The bottom graph view shows a flow diagram with nodes: 'start', '맞집 추천 ...', '벡터 검색 (Supabase pgvector)', '답변 생성', and 'end'.

## 한 줄로 계측 — @observe

▶ 중첩 함수는 자동으로 부모-자식 트레이스 — 짧은 스크립트는 flush 필수

```
from langfuse import observe, get_client

@observe()                                # 이 한 줄로 입력·출력·지연 자동 캡처
def ask(question):
    ...                                   # LLM 호출
    return answer

ask("안녕?")
get_client().flush()                     # 짧게 도는 스크립트는 flush 필수
```

## 토큰·비용 자동 + 품질 점수 남기기

▶ Score가 쌓이면 20차시 평가로 바로 이어진다

```
from langfuse.openai import openai # 이 import만 바꾸면 토큰·비용 자동
from langfuse import observe, get_client

@observe()
def ask(q):
    r = openai.chat.completions.create(model="gpt-4o-mini",
                                       messages=[{"role": "user", "content": q}])
    get_client().score_current_trace(
        name="useful", value=1, data_type="NUMERIC") # 품질 점수
    return r.choices[0].message.content
```

## Cloud vs Self-host

▶ 둘 다 같은 SDK·같은 @observe 코드 — LANGFUSE\_HOST 값 하나만 다르다

### Cloud — 빠른 시작

- cloud.langfuse.com (EU/US)
- 가입 후 키만 발급 → 즉시
- 인프라 관리 불필요

VS

### Self-host — 데이터 통제

- Docker로 직접 호스팅
- 데이터 주권·온프레미스
- 보안·규제 산업에 적합 — 캠프 기본

# 가입 → 프로젝트 생성 → API 키 발급

▶ 로컬도 클라우드와 같은 흐름 — 발급된 키 3개(SECRET·PUBLIC·HOST)를 .env에 넣으면 끝

 **Create new account**

**Name**

**Email**

**Password**

**Sign up**

Already have an account? [Sign in](#)

langfuse v3.210.0 OSS

Vibe Coding Workshop / test-blog-matijp

Go to...

Home

Dashboards

Observability

Tracing

Sessions

Users

Prompt Management

Prompts

Playground

Evaluation

Scores

Evaluators

Human Annotation

Datasets

Settings

Book a call

Support

위강 워크샵 강사 workshop@exam...

**Project Settings**

General

**API Keys**

MCP & CLI

LLM Connections

Model Definitions

Scores Configs

Members

Integrations

Exports

Batch Actions

Audit Logs

Notifications

Organization Settings

**Project API Keys**

+ Create new API keys

.env

LANGFUSE\_SECRET\_KEY="sk-lf-..."  
LANGFUSE\_PUBLIC\_KEY="pk-lf-..."  
LANGFUSE\_BASE\_URL="http://localhost:3000"

| Created      | Created By | Note              | Public Key        | Secret Key    |
|--------------|------------|-------------------|-------------------|---------------|
| 2026. 7. 10. | 워크샵 강사     | Click to add note | pk-lf-6f8af60c... | sk-lf-...777c |

17차시 · LLMOps 개요 & 라이프사이클



SECTION 06

# 정리

## 오늘 3줄

- LLMOps = 배포가 아니라 운영 — 모니터링·평가·최적화로 '불을 켜 둔다'
- 목표 4축 = 보안·확장성·견고성·신뢰성 (SLO/SLA/KPI로 숫자화), 적응은 프롬프트·RAG 먼저
- 운영의 1번 토대는 관찰성 — @observe() 한 줄로 트레이싱 시작

**보이지 않으면 고칠 수 없다.**

— 관찰성이 LLMOps의 첫 번째 토대 — 기록이 곧 운영의 시작